GUÍA RÁPIDA 2 · ENUNCIADO → SOLUCIÓN

CorporateDir WPF Directorio Corporativo

Cada paso de la **Práctica Guiada WPF MVVM Toolkit** junto a lo que tienes que hacer en Visual Studio y el código final, ya **compilado y probado** (0 errores, 0 advertencias).

Cómo leer esta guía

ENUNCIADO · Paso X

// Lo que pide el PDF del profesor.

✓ **Comprobación**
Cómo saber que el paso está bien antes de seguir.

Entre ambos: los clics en Visual Studio y el código. Las líneas resaltadas son las que responden a lo que pide el enunciado.

Introducción

+ **Estructura del proyecto**
WPF .NET 8 · NuGet · carpetas · `Sección 0`

{ } **Fase 1 · Dominio y servicio**
Empleado, IEmpleadoService, mockup · `Pasos 1.1–1.3`

▶ **Fase 2 · ViewModel y XAML**
Toolkit, 3 filas, DataContext · `Pasos 2.1–2.4`

⇩ **Entregable + revisión de tu WpfApp1**
ZIP sin bin/obj, captura y qué corregir

≈ ¿Hiciste la Guía 1? Es el mismo ejercicio con otros nombres

TechStore (Guía 1)		CorporateDir (este enunciado)	Ojo
`Producto` · `IProductoService` · `ProductoService`	→	`Empleado` · `IEmpleadoService` · `EmpleadoService`	Método: `ObtenerEmpleadosMockup()`
`Categoria` · `Precio` · `Disponible`	→	`Departamento` · `Salario` · `JornadaCompleta`	Salario con `{0:C}`
`NuevoNombre` · `NuevaCategoria` · `NuevoDisponible`	→	`NuevoNombre` · `NuevoDepartamento` · `NuevoActivo`	**NuevoActivo** = CheckBox de jornada
Grid de **2 columnas** (formulario a la izquierda)	→	Grid de **3 filas** (formulario horizontal arriba)	Cambia el layout

00 Estructura del proyecto

ENUNCIADO · 2. Estructura del Proyecto pág. 1

// Crea una solución WPF en .NET 8 denominada **WpfApp1** e instala el paquete NuGet **CommunityToolkit.Mvvm**. Organiza las carpetas: Models, Services, ViewModels y Views.

1. Crear un proyecto nuevo → **Aplicación WPF** (C#, no «.NET Framework») → nombre `WpfApp1` → Marco **.NET 8.0** → Crear.
2. Clic dcho. WpfApp1 → Administrar paquetes NuGet... → Examinar → `CommunityToolkit.Mvvm` → Instalar.
3. Clic dcho. WpfApp1 → Agregar → Nueva carpeta ×4: `Models`, `Services`, `ViewModels`, `Views`.
4. **Arrastra** `MainWindow.xaml` a `Views` y actualiza los 3 sitios: `x:Class`, `namespace` y `StartupUri` (ver abajo).

Capturas de cada diálogo en la **Guía 1**, pasos 01–03 y 07.

Explorador de soluciones — WpfApp1

```
Solución 'WpfApp1' (1 de 1 proyecto)
  WpfApp1
    Dependencias › Paquetes › CommunityToolkit.Mvvm
    Models
      Empleado.cs                    // 1.1
    Services
      IEmpleadoService.cs            // 1.2
      EmpleadoService.cs             // 1.3
    ViewModels
      MainViewModel.cs               // 2.1 · 2.2
    Views
      MainWindow.xaml                // 2.3
        MainWindow.xaml.cs           // 2.4
    App.xaml
```

XAML App.xaml — WpfApp1 · ventana de inicio

```xaml
<Application x:Class="WpfApp1.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="Views/MainWindow.xaml">
    <Application.Resources>
    </Application.Resources>
</Application>
```

XAML WpfApp1.csproj — doble clic en el proyecto para verlo

```xaml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseWPF>true</UseWPF>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="CommunityToolkit.Mvvm" Version="8.4.0" />
  </ItemGroup>

</Project>
```

1.1 Fase 1 · Modelo Empleado

Models/Empleado.cs

ENUNCIADO · Paso 1.1 (Modelo) pág. 1

// Crea la clase **Models/Empleado.cs** con las propiedades: Id (int), Nombre (string), Departamento (string), Salario (decimal) y JornadaCompleta (bool).

Clic dcho. Models | Agregar | Clase... → nombre `Empleado.cs` → escribe:

C# Empleado.cs — WpfApp1 › Models

```csharp
namespace WpfApp1.Models
{
    public class Empleado
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public string Departamento { get; set; } = string.Empty;
        public decimal Salario { get; set; }
        public bool JornadaCompleta { get; set; }
    }
}
```

✓ **Comprobación**
Son exactamente 5 propiedades con los tipos pedidos. Cambia `internal class` (lo que pone VS por defecto) por `public class`.

1.2 Fase 1 · Interfaz del servicio

Services/IEmpleadoService.cs

ENUNCIADO · Paso 1.2 (Interfaz) pág. 1

// Define la interfaz **Services/IEmpleadoService.cs** con el contrato **List<Empleado> ObtenerEmpleadosMockup()**.

Clic dcho. Services | Agregar | Nuevo elemento... | Interfaz → `IEmpleadoService.cs`:

IEmpleadoService.cs — WpfApp1 › Services

```
1  using WpfApp1.Models;
2
3  namespace WpfApp1.Services
4  {
5      public interface IEmpleadoService
6      {
7          List<Empleado> ObtenerEmpleadosMockup();
8      }
9  }
```

En una interfaz **no** se escribe `public` en el método ni cuerpo `{ }`: solo la firma terminada en `;`. El `using WpfApp1.Models;` es necesario para que reconozca `Empleado`.

1.3 Fase 1 · Servicio mockup

Services/EmpleadoService.cs

ENUNCIADO · Paso 1.3 (Servicio Mockup) pág. 1

// Implementa **Services/EmpleadoService.cs** de forma que retorne una lista inicial con **al menos 4 registros** simulados de prueba.

Clic dcho. Services | Agregar | Clase... → `EmpleadoService.cs`. Escribe `: IEmpleadoService` y pulsa `Ctrl`+`.` → Implementar interfaz: VS crea el método vacío por ti.

EmpleadoService.cs — WpfApp1 › Services

```
1  using WpfApp1.Models;
2
3  namespace WpfApp1.Services
4  {
5      public class EmpleadoService : IEmpleadoService
6      {
7          public List<Empleado> ObtenerEmpleadosMockup()
8          {
9              return new List<Empleado>
10             {
11                 new Empleado { Id = 1, Nombre = "Laura Gómez", Departamento = "Desarrollo",
12                                Salario = 2400.00m, JornadaCompleta = true },
13                 new Empleado { Id = 2, Nombre = "Carlos Ruiz", Departamento = "Marketing",
14                                Salario = 1850.50m, JornadaCompleta = false },
15                 new Empleado { Id = 3, Nombre = "Marta Díaz", Departamento = "RRHH",
16                                Salario = 2100.00m, JornadaCompleta = true },
17                 new Empleado { Id = 4, Nombre = "Javier López", Departamento = "Soporte",
18                                Salario = 1600.00m, JornadaCompleta = true }
19             };
20         }
21     }
22 }
```

Comprobación

4 registros (el mínimo pedido). Usa `m` al final del salario (`2400.00m`): sin ella C# lo trata como `double` y da error.

!! Regla de Oro: asignación del DataContext

ENUNCIADO · Regla de Oro pág. 1

// Está **estrictamente prohibido** instanciar el ViewModel dentro del XAML. El DataContext se asigna desde el C# (code-behind) para poder inyectar el servicio por constructor.

XAML MainWindow.xaml ✕ PROHIBIDO

```xaml
<Window ... xmlns:vm="clr-namespace:WpfApp1.ViewModels">
    <Window.DataContext>
        <vm:MainViewModel/>
    </Window.DataContext>
</Window>
```

C# MainWindow.xaml.cs ✓ CORRECTO

```csharp
public MainWindow()
{
    InitializeComponent();
    IEmpleadoService servicio = new EmpleadoService();
    DataContext = new MainViewModel(servicio);
}
```

Además, la versión XAML ni siquiera compilaría: `MainViewModel` no tiene constructor vacío, necesita un `IEmpleadoService`.

2.1 Fase 2 · MainViewModel: herencia e inyección

ViewModels/MainViewModel.cs

ENUNCIADO · Paso 2.1 (MainViewModel) pág. 2

// Crea **ViewModels/MainViewModel.cs** heredando de **ObservableObject**. Inyecta **IEmpleadoService** mediante su constructor y carga los datos en un **ObservableCollection<Empleado>**.

Archivo completo (lo resaltado es lo que pide el **paso 2.1**; el resto lo explica el 2.2):

C# MainViewModel.cs — WpfApp1 › ViewModels

```csharp
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using WpfApp1.Models;
using WpfApp1.Services;

namespace WpfApp1.ViewModels
{
    public partial class MainViewModel : ObservableObject
    {
        private readonly IEmpleadoService _empleadoService;

        // Colección observable: el ListView se refresca solo al añadir
        [ObservableProperty]
        private ObservableCollection<Empleado> _empleados = new();

        // Opciones del ComboBox
        [ObservableProperty]
        private List<string> _departamentos = new() { "Desarrollo", "Marketing", "RRHH", "Soporte" };

        // Campos del formulario
        [ObservableProperty]
        [NotifyCanExecuteChangedFor(nameof(AgregarEmpleadoCommand))]
        private string _nuevoNombre = string.Empty;

        [ObservableProperty]
        private string _nuevoDepartamento = "Desarrollo";

        [ObservableProperty]
        private bool _nuevoActivo = true;

        // Fila seleccionada en el ListView
        [ObservableProperty]
        private Empleado? _empleadoSeleccionado;

        // Inyección de dependencias por constructor
        public MainViewModel(IEmpleadoService empleadoService)
        {
            _empleadoService = empleadoService;
            CargarDatos();
        }
```

```
42
43          private void CargarDatos()
44          {
45              var datos = _empleadoService.ObtenerEmpleadosMockup();
46              Empleados = new ObservableCollection<Empleado>(datos);
47          }
48
49          // Validación: el botón solo se activa si hay nombre
50          private bool PuedeAgregar => !string.IsNullOrWhiteSpace(NuevoNombre);
51
52          [RelayCommand(CanExecute = nameof(PuedeAgregar))]
53          private void AgregarEmpleado()
54          {
55              var nuevo = new Empleado
56              {
57                  Id = Empleados.Count + 1,
58                  Nombre = NuevoNombre,
59                  Departamento = NuevoDepartamento,
60                  Salario = 1500.00m,
61                  JornadaCompleta = NuevoActivo
62              };
63
64              Empleados.Add(nuevo);
65
66              // Limpiamos el formulario
67              NuevoNombre = string.Empty;
68          }
69      }
70  }
```

Pide el enunciado	Dónde está
Heredar de `ObservableObject`	Línea 9 — con `partial` (obligatorio para el Toolkit, si no: `CS0260`)
Inyectar `IEmpleadoService` por constructor	Líneas 11 y 37–41: el constructor recibe la **interfaz** y la guarda en un campo `readonly`
Cargar en `ObservableCollection<Empleado>`	Líneas 14–15 y 43–47: `CargarDatos()` pide la lista al servicio y la envuelve

2.2 Fase 2 · Propiedades reactivas y comando

ENUNCIADO · Paso 2.2 (Comandos y Propiedades) pág. 2

// Genera las propiedades reactivas **NuevoNombre**, **NuevoDepartamento** y **NuevoActivo** con **[ObservableProperty]**. Añade el método **AgregarEmpleado()** decorado con **[RelayCommand]** y validación de ejecución.

MainViewModel.cs — propiedades del formulario

```
21          // Campos del formulario
22          [ObservableProperty]
23          [NotifyCanExecuteChangedFor(nameof(AgregarEmpleadoCommand))]
24          private string _nuevoNombre = string.Empty;
25
26          [ObservableProperty]
27          private string _nuevoDepartamento = "Desarrollo";
28
29          [ObservableProperty]
30          private bool _nuevoActivo = true;
31
32          // Fila seleccionada en el ListView
33          [ObservableProperty]
34          private Empleado? _empleadoSeleccionado;
```

MainViewModel.cs — validación + comando

```
49          // Validación: el botón solo se activa si hay nombre
50          private bool PuedeAgregar => !string.IsNullOrWhiteSpace(NuevoNombre);
51
52          [RelayCommand(CanExecute = nameof(PuedeAgregar))]
53          private void AgregarEmpleado()
54          {
55              var nuevo = new Empleado
56              {
57                  Id = Empleados.Count + 1,
58                  Nombre = NuevoNombre,
59                  Departamento = NuevoDepartamento,
60                  Salario = 1500.00m,
61                  JornadaCompleta = NuevoActivo
62              };
63
64              Empleados.Add(nuevo);
65
66              // Limpiamos el formulario
67              NuevoNombre = string.Empty;
68          }
```

Escribes (campo privado)		El Toolkit genera	Se enlaza en el XAML con
`_nuevoNombre`	→	`NuevoNombre`	`TextBox`.Text
`_nuevoDepartamento`	→	`NuevoDepartamento`	`ComboBox`.SelectedItem
`_nuevoActivo`	→	`NuevoActivo`	`CheckBox`.IsChecked (jornada)
`AgregarEmpleado()` + `[RelayCommand]`	→	`AgregarEmpleadoCommand`	`Button`.Command

«Validación de ejecución» = CanExecute

`CanExecute = nameof(PuedeAgregar)` desactiva el botón si el nombre está vacío, y `[NotifyCanExecuteChangedFor(…)]` sobre `_nuevoNombre` hace que se reevalúe cada vez que el usuario teclea.

Dentro de los métodos, usa siempre las propiedades

`NuevoNombre`, no `_nuevoNombre`. Con el campo la vista no se entera y aparece el aviso `MVVMTK0034`.

2.3 Fase 2 · Vista XAML de 3 filas

Views/MainWindow.xaml

ENUNCIADO · Paso 2.3 (Vista XAML) pág. 2

// Diseña **Views/MainWindow.xaml** con una estructura vertical de 3 filas: **1.** Formulario horizontal dentro de un Border (TextBox, ComboBox de departamentos, CheckBox de jornada y Button enlazado a AgregarEmpleadoCommand). **2.** ListView con GridView (salario con {0:C}). **3.** Barra de estado: TextBlock con Run vinculado a EmpleadoSeleccionado.Nombre.

Esquema del Grid

Row 0 · Auto — Border › StackPanel Horizontal
Label | TextBox | Label | ComboBox | CheckBox | Button

Row 1 · * — ListView › GridView
ID · Nombre · Departamento · Salario {0:C} · Jornada

Row 2 · Auto — TextBlock › Run «Seleccionado: ...»

Abre `Views/MainWindow.xaml`, borra el `<Grid></Grid>` vacío de la plantilla y pega el contenido. Revisa que `x:Class` sea `WpfApp1.Views.MainWindow`.

- `Height="Auto"`: la fila mide lo que su contenido. `Height="*"`: se queda el resto.
- `Orientation="Horizontal"` en el `StackPanel` → formulario en línea.
- `UpdateSourceTrigger=PropertyChanged` → el botón se activa al teclear.
- `Language="es-ES"` → el salario sale en **€** y no en $.

XAML MainWindow.xaml × WpfApp1 › Views

```xaml
<Window x:Class="WpfApp1.Views.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="CorporateDir WPF" Height="500" Width="760"
        Language="es-ES"
        WindowStartupLocation="CenterScreen">

    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>  <!-- 1. Formulario -->
            <RowDefinition Height="*"/>     <!-- 2. Listado -->
            <RowDefinition Height="Auto"/>  <!-- 3. Barra de estado -->
        </Grid.RowDefinitions>

        <!-- FILA 0: Formulario horizontal -->
        <Border Grid.Row="0" Background="#F4F4F6" CornerRadius="8"
                Padding="12" Margin="0,0,0,12">
            <StackPanel Orientation="Horizontal">
                <Label Content="Nombre:" VerticalAlignment="Center"/>
                <TextBox Text="{Binding NuevoNombre, UpdateSourceTrigger=PropertyChanged}"
                         Width="170" Padding="5" Margin="0,0,12,0"/>

                <Label Content="Departamento:" VerticalAlignment="Center"/>
                <ComboBox ItemsSource="{Binding Departamentos}"
                          SelectedItem="{Binding NuevoDepartamento}"
                          Width="120" Padding="5" Margin="0,0,12,0"/>

                <CheckBox Content="Jornada completa"
                          IsChecked="{Binding NuevoActivo}"
                          VerticalAlignment="Center" Margin="0,0,12,0"/>

                <Button Content="Agregar"
                        Command="{Binding AgregarEmpleadoCommand}"
                        Padding="14,6" Background="#007ACC"
                        Foreground="White" FontWeight="Bold"/>
            </StackPanel>
        </Border>

        <!-- FILA 1: Listado principal -->
        <ListView Grid.Row="1"
                  ItemsSource="{Binding Empleados}"
                  SelectedItem="{Binding EmpleadoSeleccionado}">
            <ListView.View>
                <GridView>
                    <GridViewColumn Header="ID" Width="40"
                                    DisplayMemberBinding="{Binding Id}"/>
```

```
                    <GridViewColumn Header="Nombre" Width="180"
                                    DisplayMemberBinding="{Binding Nombre}"/>
                    <GridViewColumn Header="Departamento" Width="130"
                                    DisplayMemberBinding="{Binding Departamento}"/>
                    <GridViewColumn Header="Salario" Width="110"
                                    DisplayMemberBinding="{Binding Salario, StringFormat={}{0:C}}"/>
                    <GridViewColumn Header="Jornada" Width="80"
                                    DisplayMemberBinding="{Binding JornadaCompleta}"/>
                </GridView>
            </ListView.View>
        </ListView>

        <!-- FILA 2: Barra de estado -->
        <TextBlock Grid.Row="2" Margin="0,10,0,0" FontWeight="SemiBold">
            <Run Text="Seleccionado: "/>
            <Run Text="{Binding EmpleadoSeleccionado.Nombre, FallbackValue='Ninguno'}"
                 Foreground="#007ACC" FontWeight="Bold"/>
        </TextBlock>
    </Grid>
</Window>
```

2.4 Fase 2 · Code-behind

Views/MainWindow.xaml.cs

ENUNCIADO · Paso 2.4 (Code-Behind) pág. 2

> // Instancia el servicio y el ViewModel dentro de **MainWindow.xaml.cs** (el enunciado da el código del constructor).

MainWindow.xaml.cs — WpfApp1 › Views

```
using System.Windows;
using WpfApp1.Services;
using WpfApp1.ViewModels;

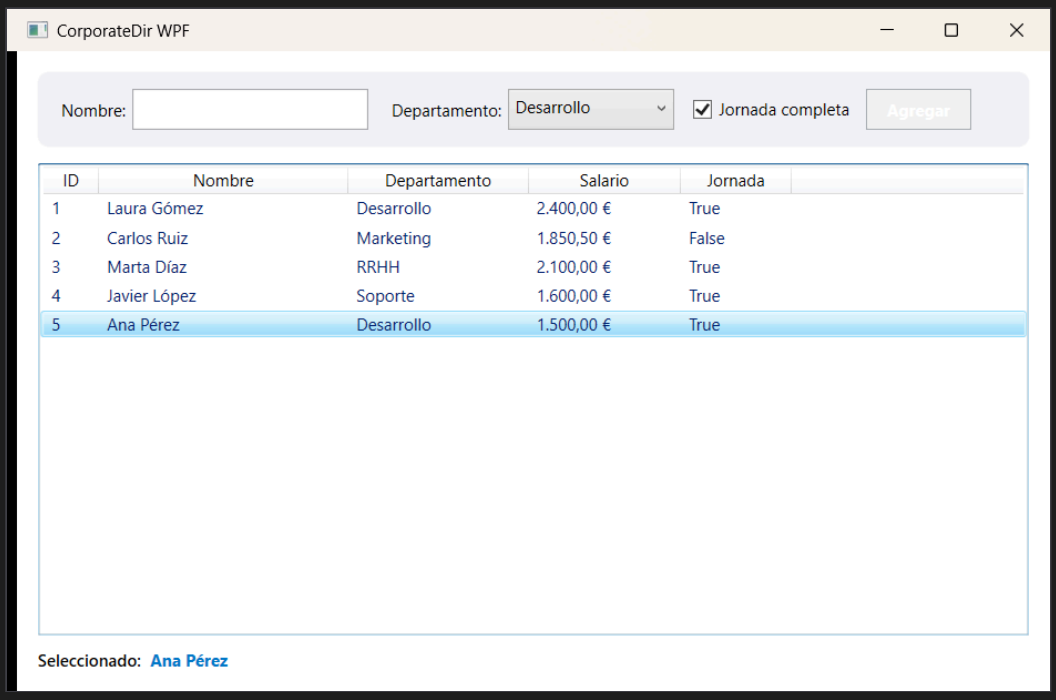
namespace WpfApp1.Views
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            IEmpleadoService servicio = new EmpleadoService();
            DataContext = new MainViewModel(servicio);
        }
    }
}
```

Los 2 `using` y el namespace

Sin `using WpfApp1.Services;` y `using WpfApp1.ViewModels;` sale `CS0246` (pulsa `Ctrl`+`.` y VS los añade). Si el `namespace` no es `WpfApp1.Views` sale `CS0103 'InitializeComponent'`.

Resultado: pulsa F5



CorporateDir en ejecución tras añadir «Ana Pérez» (ID 5) y seleccionarla. El botón aparece gris porque el nombre ya se ha vaciado.

Salida — Mostrar salida de: Compilación

```
1>------ Operación Compilar iniciada: proyecto: WpfApp1, configuración: Debug Any CPU ------
1>WpfApp1 -> C:\Users\alumno\Desktop\WpfApp1\bin\Debug\net8.0-windows\WpfApp1.dll
========== Compilar: 1 correcto, 0 incorrectos, 0 actualizado, 0 omitido ==========
```

Entregable requerido

ENUNCIADO · Entregable Requerido pág. 2

// Comprime la carpeta del proyecto **omitiendo los directorios bin y obj**. Adjunta una captura de pantalla donde se observe el formulario **agregando un registro** dinámicamente a la tabla.

1. Con la app en marcha (`F5`), escribe un nombre, pulsa **Agregar** y selecciona la nueva fila.
2. Captura con `Win` + `Mayús` + `S` → modo ventana → pega en Paint y guarda `captura.png`.
3. **Cierra Visual Studio** (si no, bloquea `obj`). En el Explorador de soluciones antes puedes usar Clic dcho. WpfApp1 → Abrir carpeta en el Explorador de archivos.
4. Copia la carpeta del proyecto y en la copia **borra** `bin`, `obj` (y `.vs`, carpeta oculta).
5. Clic dcho. carpeta → Comprimir en archivo ZIP y entrega el ZIP + `captura.png`.

> «Limpiar solución» de VS **no** borra `obj`: hay que eliminarlas a mano.

Revisión de tu WpfApp1 actual (carpeta del Escritorio)

Tu proyecto **compila y funciona**, pero estos puntos no cumplen el enunciado o dan avisos. Ordenados por importancia:

Lista de errores — 3 No cumple · 2 Advertencias · 4 Mejoras

	Archivo	Qué pasa	Cómo arreglarlo
✕	`MainWindow.xaml`	Está en la raíz: no existe la carpeta `Views`.	Crea `Views`, arrastra la ventana, cambia `x:Class`, `namespace` y `StartupUri="Views/MainWindow.xaml"`.
✕	`MainWindow.xaml`	Grid de 2 **columnas** (formulario a la izquierda).	El enunciado pide 3 **filas** y formulario horizontal: usa el XAML del paso 2.3.
✕	`MainViewModel.cs`	La propiedad del CheckBox se llama `_JornadaCompleta`.	Renómbrala a `_nuevoActivo` → genera `NuevoActivo`, y en el XAML `IsChecked="{Binding NuevoActivo}"`.
⚠	`MVVMTK0034 ×4`	`AgregarEmpleado()` y `_CanAgregar` leen los campos `_NuevoNombre`, `_NuevoDepartamento`…	Usa las propiedades: `NuevoNombre`, `NuevoDepartamento`, `NuevoActivo`.
⚠	`MainViewModel.cs`	`_NuevoDepartamento = string.Empty`: el ComboBox arranca vacío y el empleado puede quedar sin departamento.	Inicialízalo con un valor de la lista, p. ej. `"Lengua"`.
i	`MainViewModel.cs`	`_NuevoSalario` se declara pero no se usa (salario fijo `2300m`).	Bórralo, o añade un TextBox `{Binding NuevoSalario}` y usa `Salario = NuevoSalario`.
i	`models / services`	Namespaces en minúscula (`WpfApp1.models`).	Convención: `WpfApp1.Models`, `WpfApp1.Services` (como las carpetas).
i	`WpfApp1.csproj`	`net10.0-windows`; el enunciado dice .NET 8.	Si el PC del examen solo tiene .NET 8: `<TargetFramework>net8.0-windows</TargetFramework>`.
i	`MainWindow.xaml`	Salario sale como **$2,000.00**; columna «Jornada Completa» de 60 px se corta.	Añade `Language="es-ES"` al `<Window>` y da más ancho a la columna.

Checklist final (lo que mirará el profesor)

- [ ] Proyecto WPF **.NET 8** llamado `WpfApp1` con `CommunityToolkit.Mvvm`.
- [ ] Carpetas `Models`, `Services`, `ViewModels`, `Views`.
- [ ] `Empleado` con Id, Nombre, Departamento, Salario, JornadaCompleta.
- [ ] `IEmpleadoService` + `EmpleadoService` con ≥ 4 registros.
- [ ] VM `partial` que hereda de `ObservableObject` y recibe la interfaz.
- [ ] `NuevoNombre`, `NuevoDepartamento`, `NuevoActivo` con `[ObservableProperty]`.
- [ ] `[RelayCommand]` con `CanExecute` (botón gris si no hay nombre).
- [ ] XAML de 3 filas: Border horizontal · ListView + GridView `{0:C}` · Run con el seleccionado.
- [ ] **Ningún** ViewModel en el XAML; DataContext en el code-behind.
- [ ] ZIP sin `bin` / `obj` + captura añadiendo un registro.